
Taller de Programación Avanzada

Sistema de Subastas Concurrentes "Órbita Alfa Salvage"

1. Contexto del Sistema (Caso de Negocio)

La estación espacial **Órbita Alfa** ha centralizado el mercado de recuperación de materiales y chatarra estelar del sector. Los recolectores traen componentes recuperados de naves destruidas y los venden al mejor postor mediante un portal web.

Hasta ahora, los sistemas de la estación procesaban los datos de forma secuencial (uno detrás de otro). Sin embargo, en un entorno de alta concurrencia donde múltiples comerciantes pujan por el mismo recurso al mismo tiempo, el sistema viejo genera cuellos de botella y errores en las cuentas de créditos de los usuarios.

Para resolver esto, la estación requiere un nuevo **Motor de Subastas Asíncrono**. El objetivo principal es recibir las ofertas mediante un formulario web, procesar las validaciones en paralelo para ahorrar tiempo de cómputo y actualizar las tablas de forma masiva y simultánea.

2. Arquitectura de Datos (Persistencia en MySQL)

El sistema del mercado negro espacial opera bajo dos estructuras de datos relacionadas en el motor relacional de la estación:

1. **Tabla de Lotes:** Almacena la descripción del material, la puja* económica más alta registrada hasta el momento y el nombre del comerciante que va liderando la subasta.
2. **Tabla de comerciantes:** Almacena el nombre único de cada usuario y su saldo disponible en Créditos Estelares.

¿Qué es una Puja? Una puja (u oferta) es la cantidad de dinero que un comprador está dispuesto a pagar por algo que se está subastando. Es la acción de alzar la mano y decir: "Yo ofrezco X cantidad de dinero por ese objeto".

La regla de oro: Para que tu puja sea válida, debe ser estrictamente mayor que la última puja que ofreció otra persona. Si no es mayor, el sistema la rechaza de forma automática.

Datos Iniciales de la Base de Datos:

- **Lote 1:** Restos de Fuselaje de Titanio | Puja Actual: 500 créditos.
- **Lote 2:** Núcleo de Hiperpropulsor Dañado | Puja Actual: 1200 créditos.
- **Comerciante A:** HanSolo | Saldo: 2000 créditos.
- **Comerciante B:** Watto | Saldo: 800 créditos.
- **Comerciante C:** Nebula9 | Saldo: 5000 créditos.

3. Requerimientos Técnicos del Proyecto

Deben crear una estructura limpia con dos archivos en la raíz y una carpeta pública para la interfaz:

```
orbita_alfa_subastas/  
├── public/  
│   ├── css/  
│   │   └── estilos.css  
│   └── index.html  
├── db.js  
└── server.js
```

El proyecto debe estar modularizado y estructurado utilizando la arquitectura de software de **Express**, separando los recursos públicos del servidor central.

A. La Interfaz de Usuario (Frontend: HTML5 + CSS3)

- Cree un directorio llamado `public`. Dentro de él, configure un único archivo HTML conectado a una hoja de estilos externa (`estilos.css`).
- El diseño visual debe ser limpio y básico, enfocado en componentes de tarjetas (`.card`) utilizando el modelo de caja elemental (márgenes, rellenos y bordes sólidos). **Nota: Queda completamente prohibido el uso de herramientas de IA para la generación del código de diseño.**
- Debe construir un **Formulario de Ofertas** que capture tres datos obligatorios:
 1. El ID numérico del lote por el que se desea competir (atributo `name="idLote"`).
 2. El nombre de usuario del comerciante que realiza la puja (atributo `name="comerciante"`).

3. El monto económico de la oferta en créditos (atributo `name="oferta"`).

- El formulario debe despachar los datos utilizando el método HTTP correcto para el envío de información de negocio hacia el backend.
- **Botón de Monitoreo:** Debajo del formulario, agregue un enlace o botón independiente que redirija al usuario a la ruta `/consultar` para verificar el estado de las tablas.

B. El Controlador (Backend: Node.js + Express + Promesas)

- **Middleware de Lectura:**
 - Configure su servidor para poder interceptar y traducir los datos que vienen del formulario web hacia el objeto `req.body`, evitando lecturas `undefined`.
- **Ruta de Procesamiento (POST):**
 - Cree el endpoint `app.post('/subasta')` que reciba el formulario. Dentro de esta ruta, debe aplicar asincronía. Para evitar retrasos en el servidor, las consultas de lectura a la base de datos (verificar el lote y verificar el saldo del comerciante) deben dispararse **al mismo tiempo en paralelo** utilizando herramientas de control de flujo asíncrono de JavaScript (`Promise.all`).
 - Si todo es correcto, actualiza los datos y responde con un mensaje de éxito básico.
 - Las consultas de actualización (restar los créditos al usuario y cambiar el líder de la subasta) también deben ejecutarse simultáneamente solo si la oferta es válida.
- **Ruta de Reporte con Parámetros (GET):**
 - Cree el endpoint `app.get('/consultar')`. Cuando el usuario presione el botón de consultar en la página principal, esta ruta debe usar `Promise.all` para realizar dos consultas simultáneas a la base de datos: un `SELECT` para traer todos los lotes y otro `SELECT` para traer todos los comerciantes. El servidor tomará ambos resultados y los pintará en dos tablas HTML sencillas para mostrar el estado en vivo del mercado.

4. Lógica de Negocio y Control de Escenarios (Escenarios a Validar)

El controlador del backend debe analizar el resultado de las promesas paralelas y validar de forma estricta los siguientes escenarios antes de aprobar cualquier

transacción:

- **Escenario de Error 1 (Datos Inexistentes):** Si el ID del lote consultado no existe en la base de datos, o el nombre del comerciante no está registrado en el sistema, el servidor debe detener la operación de inmediato y devolver una pantalla con un mensaje de advertencia claro.
- **Escenario de Error 2 (Viabilidad Financiera):** El sistema debe comparar la oferta económica contra los fondos actuales del comerciante. Si el comerciante intenta pujar por un monto mayor a los créditos que posee, la operación debe ser rechazada.
- **Escenario de Error 3 (Regla de Competencia):** La nueva oferta debe ser **estrictamente mayor** a la puja actual registrada en el lote. Si el monto es menor o igual, el sistema debe bloquear el intento.
- **Escenario de Éxito (Transacción Atómica):** Si el envío supera con éxito todas las validaciones anteriores, el sistema debe proceder a actualizar las tablas en paralelo (descontar los créditos correspondientes al comerciante y registrarlo como el nuevo líder del lote con su respectivo monto). Finalmente, debe retornar una interfaz limpia al usuario confirmando que la operación fue un éxito.

5. Criterios de Aceptación (Flujo de Pruebas)

Para comprobar que el sistema cumple con los estándares de ingeniería requeridos, el laboratorio se evaluará ejecutando el siguiente flujo de extremo a extremo:

1. Se levantará el servidor local en el puerto determinado empleando un monitor de cambios en tiempo real (nodemon).
2. Desde el navegador web, se completará el formulario simulando un fallo de fondos (ej: *Watto* ofertando 900 créditos por el lote 1). El sistema no debe caerse; debe mostrar el error financiero de forma controlada.
3. Se realizará una puja exitosa (ej: *HanSolo* ofertando 600 créditos por el lote 1). El sistema debe redirigir a la pantalla de éxito.
4. **Verificación de Persistencia:** Al hacer clic en el botón de consultar (o ingresar manualmente a la ruta `/consultar`), los cambios deben verse reflejados inmediatamente en las tablas impresas (el lote 1 debe mostrar a HanSolo como líder con 600 créditos, y el saldo de HanSolo en la tabla de comerciantes debe haber disminuido de manera proporcional a 1400 créditos).